TIVIDY - UML Class Diagram (Task 5, revision 2: fully generic)

COS 214 Practical 6 - generic workflow management system. Every class, attribute and operation is scenario-independent: a concrete process (such as our demonstration scenario) is supplied only as data through a WorkflowDirector (definition file or interactive input), never as classes in the engine.

Contents (vector pages - zoom in for full detail)

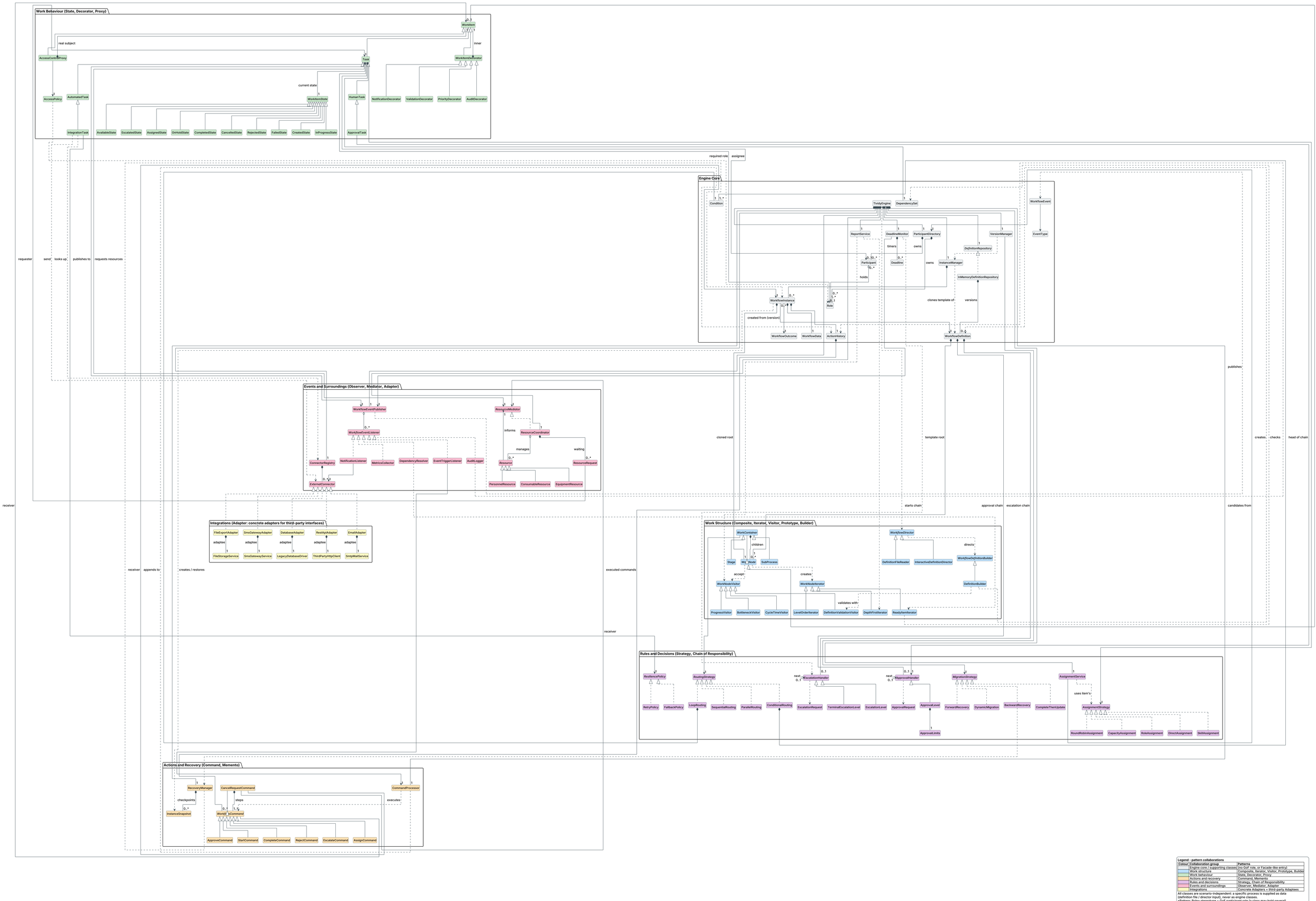
Figure 5.0 Complete class diagram - all classes and relationships (attributes and operations hidden)
Figure 5.1 Engine core: definitions, instances, participants and engine-owned services
Figure 5.2 Work structure: Composite, Iterator, Visitor, Prototype, Builder
Figure 5.3 Work behaviour: State, Decorator, Proxy
Figure 5.4 Actions and recovery: Command, Memento
Figure 5.5 Rules and decisions: Strategy, Chain of Responsibility
Figure 5.6 Events and surroundings: Observer, Mediator, Adapter (with the generic integration adapters)
Figure 5.7 Complete class diagram with all attributes and operations

Legend - pattern collaborations (repeated on every figure)

Engine core / supporting classes	no GoF role, or Facade-like entry point
Work structure	Composite, Iterator, Visitor, Prototype, Builder
Work behaviour	State, Decorator, Proxy
Actions and recovery	Command, Memento
Rules and decisions	Strategy, Chain of Responsibility
Events and surroundings	Observer, Mediator, Adapter
Integrations	Concrete Adapters and third-party Adaptees

<<Pattern: Role>> stereotype = GoF participant role; a class may play several roles.
Filled diamond = composition (owner deletes the part); hollow diamond = aggregation (shared, not deleted).
Solid arrow = navigable association; dashed arrow = dependency (uses / creates); hollow triangle = inheritance (dashed = realisation).

Figure 5.0 - TIVIDY generic workflow engine: complete class diagram (members shown in Figures 5.1-5.6)



Legend - pattern collaborations

Colour	Collaboration group	Patterns
	Engine core / supporting classes	(no GoF role, or Facade-like entry)
	Work structure	Composite, Iterator, Visitor, Prototype, Builder
	Work behaviour	State, Decorator, Proxy
	Actions and recovery	Command, Memento
	Rules and decisions	Strategy, Chain of Responsibility
	Events and surroundings	Observer, Mediator, Adapter
	Integrations	Concrete Adapters + third-party Adaptees

All classes are scenario-independent: a specific process is supplied as data (definition file / director input), never as engine classes.
«Pattern: Role» stereotype = GoF participant role (a class may hold several).
Filled diamond = composition (owner deletes part); hollow diamond = aggregation (shared).
Dashed arrow = dependency (uses / creates); solid arrow = navigable association.

Figure 5.1 - Engine core: definitions, instances, participants and engine-owned services

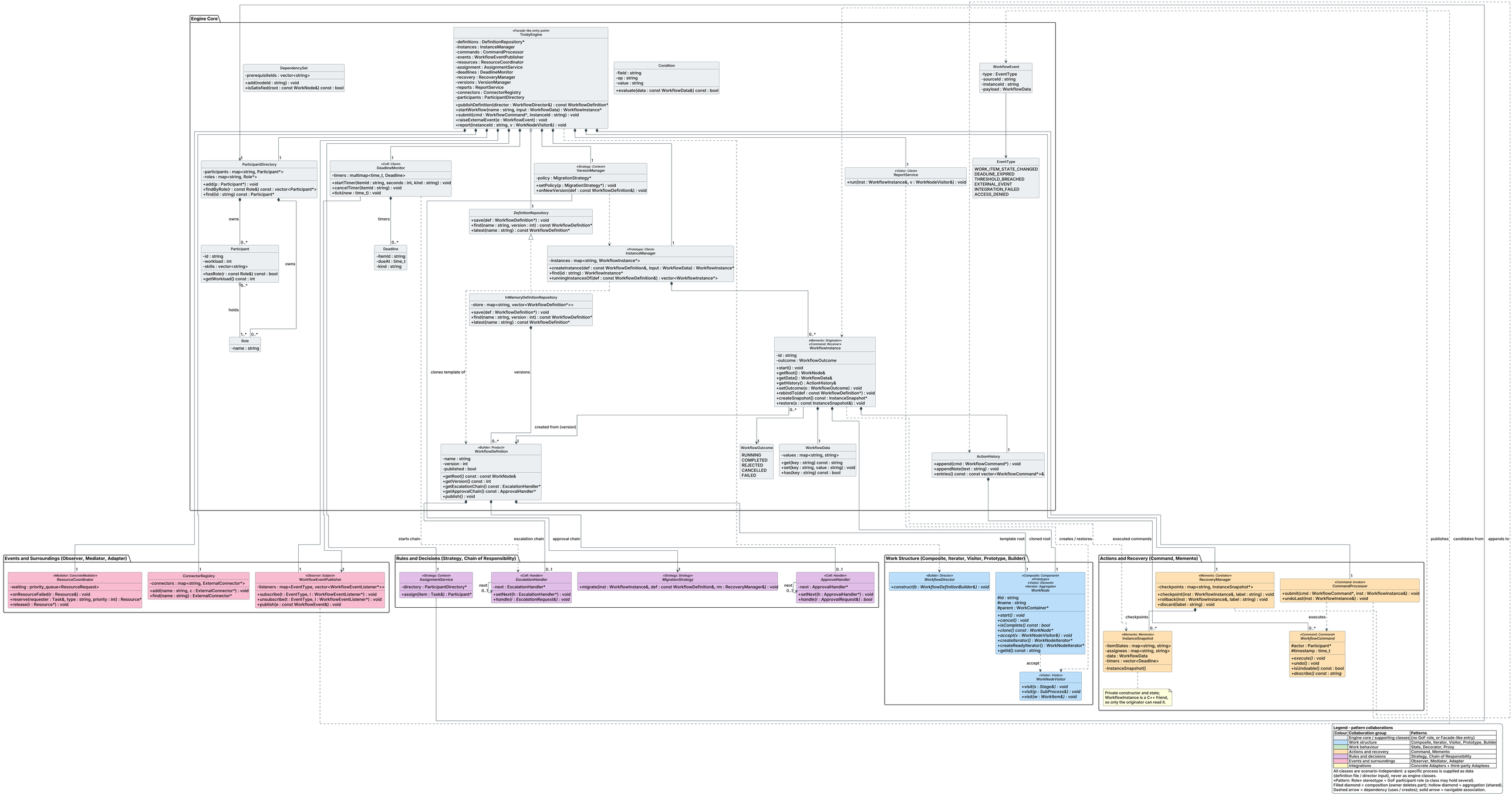


Figure 5.2 - Work structure: Composite, Iterator, Visitor, Prototype, Builder

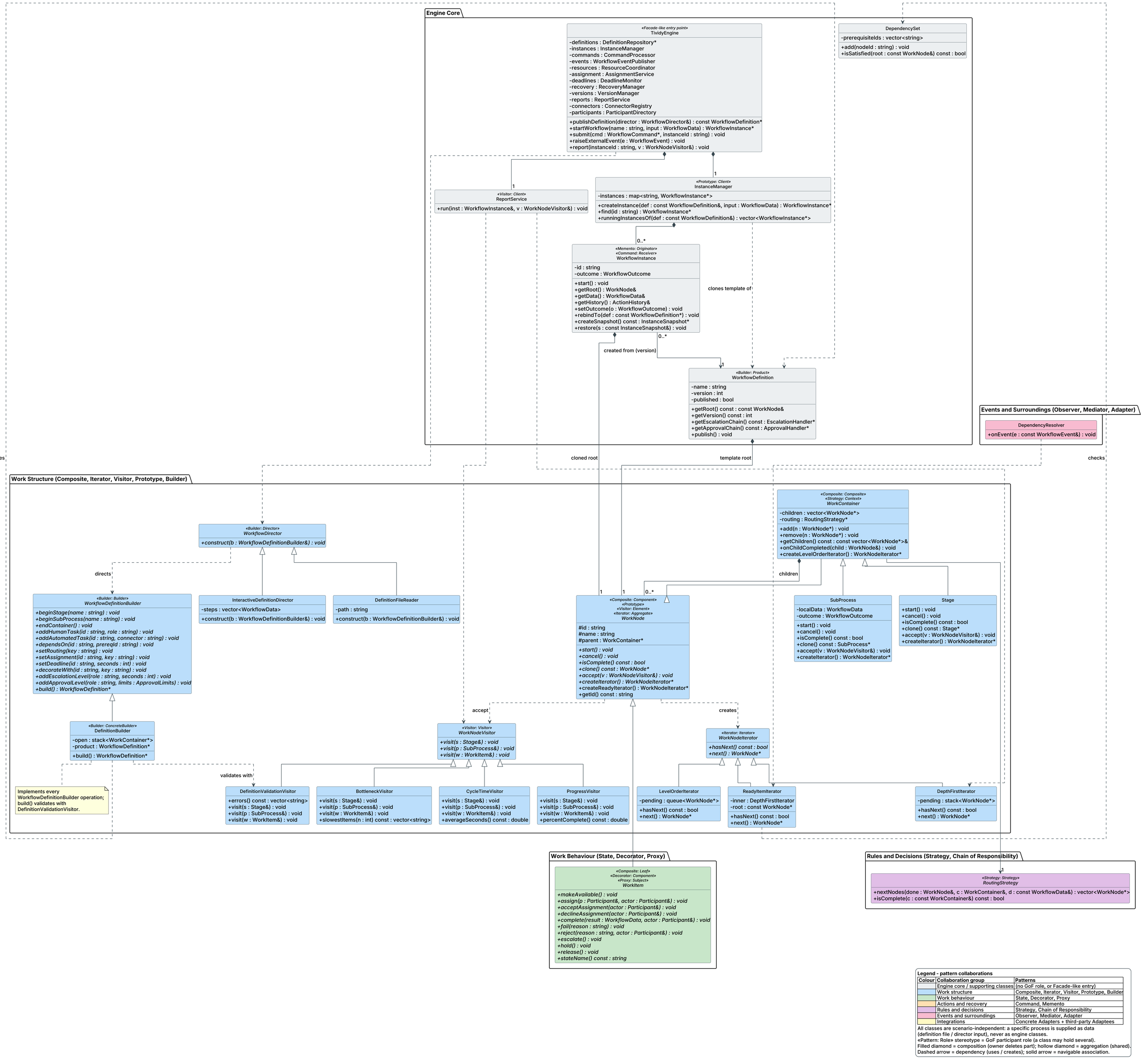


Figure 5.3 – Work behaviour: State, Decorator, Proxy

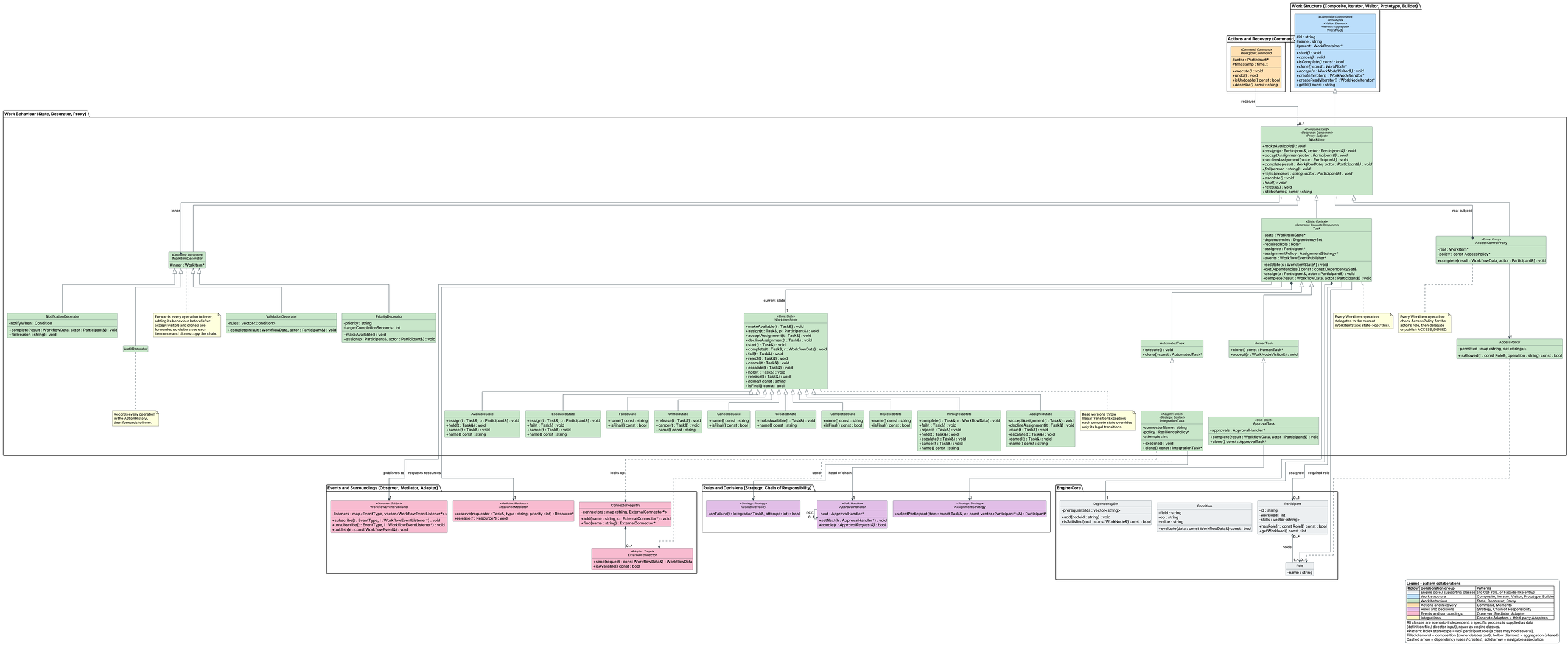
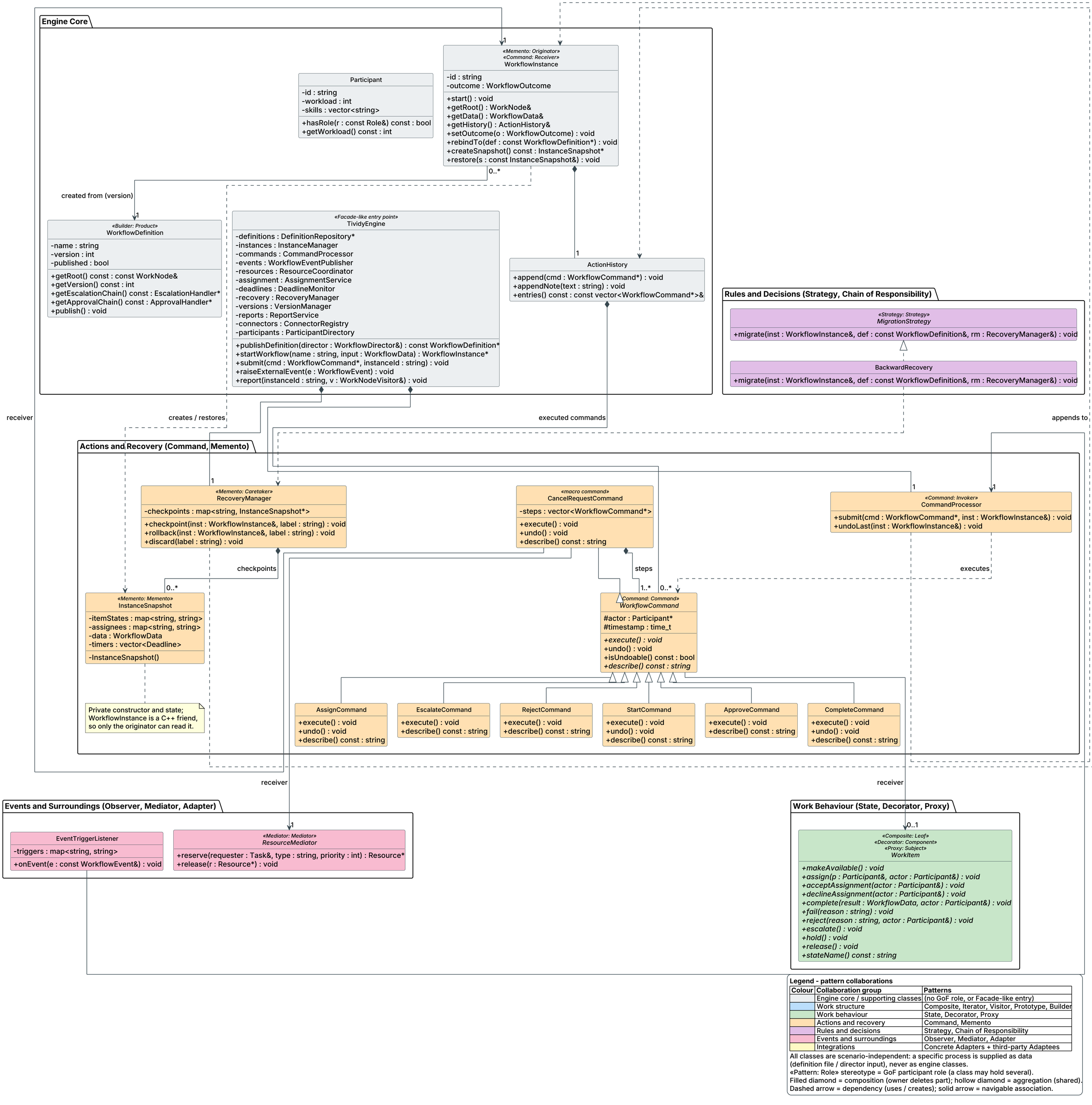


Figure 5.4 - Actions and recovery: Command, Memento



Legend - pattern collaborations

Colour	Collaboration group	Patterns
	Engine core / supporting classes	(no GoF role, or Facade-like entry)
	Work structure	Composite, Iterator, Visitor, Prototype, Builder
	Work behaviour	State, Decorator, Proxy
	Actions and recovery	Command, Memento
	Rules and decisions	Strategy, Chain of Responsibility
	Events and surroundings	Observer, Mediator, Adapter
	Integrations	Concrete Adapters + third-party Adaptees

All classes are scenario-independent: a specific process is supplied as data (definition file / director input), never as engine classes.
«Pattern: Role» stereotype = GoF participant role (a class may hold several).
Filled diamond = composition (owner deletes part); hollow diamond = aggregation (shared).
Dashed arrow = dependency (uses / creates); solid arrow = navigable association.

Figure 5.5 – Rules and decisions: Strategy, Chain of Responsibility

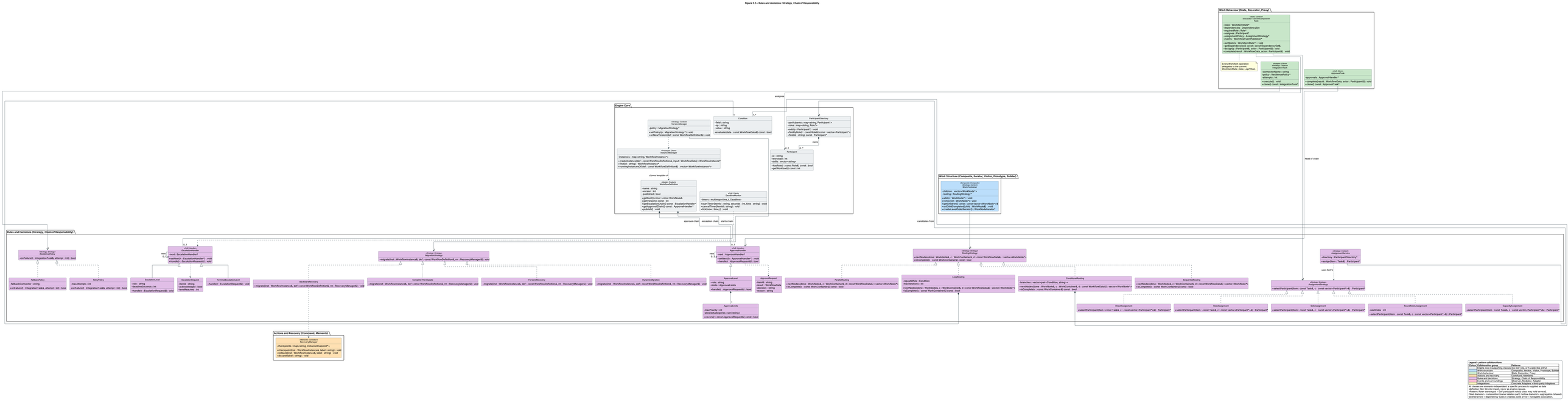


Figure 5.6 - Events and surroundings: Observer, Mediator, Adapter

